



Sistema de Gestión de Biblioteca

Trabajo Final de Programación



Universidad Europea de Valencia



Ingeniería Informática



Integrantes del Grupo

Vladyslav Taratunskyi • Alejandro Castro • Álvaro Rovira • Enrique Olmo



Primer Curso · Curso 2025-2026

El Proyecto: Reto Biblioteca POO

¿Qué es?



Catálogo

Libros, dispositivos, juegos y recursos digitales



Usuarios

Socios y empleados con roles diferenciados



Préstamos

Ciclo completo automatizado con sanciones

Objetivos

1

Sistema Modular y Mantenable

Separación de responsabilidades

2

Jerarquía de Clases

Herencia para especialización sin duplicar

3

Préstamos Automáticos

Fechas y sanciones calculadas automáticamente

4

Diseño Extensible

Fácil de añadir nuevos tipos de materiales

 **Por qué POO:** Modela mejor la realidad del dominio bibliotecario y facilita la evolución del sistema

Arquitectura de Tres Capas y Pilares POO

◆ Arquitectura de Tres Capas

Lógica de Negocio

models.py: Reglas del dominio,
independiente de BD e interfaz

Persistencia

Base de datos: Almacenamiento y
recuperación de datos

Interfaz

Interacción con el usuario,
presentación visual

 Principio de Separación de Responsabilidades: Cambiar BD o interfaz no afecta la lógica de negocio

🧩 Pilares POO Aplicados

Encapsulamiento

Atributos privados con getters/setters validados

Herencia

Material → MaterialFisico → Libro/Dispositivo/Juego

Polimorfismo

prestar() funciona igual para todos los materiales

Abstracción

Clase abstracta Material obliga a respetar interfaz

Arquitectura de Clases: Jerarquía y Relaciones



Colores por Rama

■ Material (ABC) ■ Usuario (ABC) ■ Préstamo

Decisiones de Diseño

Separación de comportamientos físicos vs digitales y socios vs empleados mediante herencia jerárquica

Decisiones Técnicas Clave

Clases Abstractas

Material y Usuario son **ABC**:
obligan a implementar métodos
comunes

- ✓ Interfaz común garantizada
- ✓ Previenen errores en desarrollo
- ✓ Fuerzan a respetar el contrato
- ✓ Documentan la intención

Métodos obligatorios: prestar(),
devolver(), puede_devolverse(),
descripcion_corta()

Enumeradores

Valores fijos y controlados en
lugar de strings sueltos

- ✓ Eliminan errores de tipo
- ✓ Autocompletado en el IDE
- ✓ Centralización de valores

Enums: EstadoMaterial,
EstadoPrestamo, TipoDispositivo,
RolEmpleado

Properties

Getters/setters con **@property**
validan antes de asignar

- ✓ Validación en el modelo, no en la interfaz
- ✓ Funciona con cualquier fuente de datos
- ✓ Ejemplos: email con arroba, jugadores min ≤ max

Principio: La validación pertenece al
modelo, no a la pantalla

Usuarios y Control de Acceso

SOCIO

-  Pide préstamos
-  Límite de préstamos simultáneos
-  Control de sanciones
-  Permiso VIP/investigadores

Nunca gestiona catálogo

EMPLEADO

-  Gestiona catálogo y usuarios
-  Rol asignado (Bibliotecario, Auxiliar, Admin)
-  Métodos centralizados de permisos
-  Nunca pide préstamos

Principio ISP: Evita métodos inútiles en cada clase

Tres Niveles de Permiso

1

Auxiliar

Operaciones básicas

2

Bibliotecario

Operaciones avanzadas + bloqueo

3

Administrador

Operaciones críticas y destructivas

✓ Métodos centralizados: `es_admin()`, `es_bibliotecario_o_superior()`

Sistema de Préstamos y Sanciones

↔ Préstamos Automatizados

Clase **Prestamo**: contrato entre socio y material con gestión automática del ciclo completo

 **Libros**: 15 días

 **Dispositivos**: 7 días

 **Juegos de mesa**: 2 días

 Renovación solo si préstamo activo



ConfiguracionBiblioteca: Valores centralizados para fechas de préstamo y sanciones. Un solo cambio actualiza todo el sistema.

Sanciones Automáticas

Detección automática de retrasos y aplicación de sanciones sin intervención humana

✓ Sin errores humanos

✓ Sin olvidos

✓ Sin favoritismos

✓ Consistencia total

⊘ Socio bloqueado hasta levantar sanción

Gestión de Estados y Configuración

Materiales Físicos

Estados secuenciales: Disponible → Reservado → Prestado



Disponible →



Reservado →



Prestado

Recursos Digitales

Pool de licencias concurrentes: cálculo disponibles/totales



Cálculo automático de licencias



Bloqueo manual por bibliotecario



Decisión manual tiene prioridad absoluta

Configuración Biblioteca



Días de préstamo



Máximo préstamos



Valores de sanciones



Límites renovaciones



Cuota VIP/investigadores



Fuente Única de Verdad



Principio DRY: Un solo cambio actualiza todo el sistema

Ventajas, Desafíos y Conclusiones

Ventajas

- ✓ **Mantenibilidad:** Cambios localizados
- ✓ **Escalabilidad:** Crece sin romperse
- ✓ **Testabilidad:** Fácil de probar
- ✓ **Extensibilidad:** Nuevos tipos fácilmente
- ✓ **Robustez:** Validación en modelo
- ✓ **Flexibilidad:** Cambia BD o interfaz

Desafíos y Soluciones

Problema: Lógica mezclada

Solución: Herencia separada

Problema: Estados digitales complejos

Solución: `actualizar_estado()` respeta bloqueo

Problema: Validación duplicada

Solución: Properties en modelo

Problema: Constantes dispersas

Solución: ConfiguraciónBiblioteca

Problema: Permisos dispersos

Solución: Métodos centralizados

Conclusiones

- ✓ POO modela la realidad
- ✓ Decisiones de diseño > código
- ✓ Separación de responsabilidades
- ✓ Clases abstractas previenen errores
- ✓ Validación en modelo, no interfaz
- ✓ Configuración centralizada
- ✓ Polimorfismo reduce acoplamiento



Resultado final: Un sistema que no solo funciona hoy, sino que está preparado para crecer, adaptarse y mantenerse con el mínimo esfuerzo.



¡GRACIAS!

Por vuestra atención



Universidad Europea de Valencia



Ingeniería Informática



Primer Curso · 2025-2026



Demo del Programa

